

CP Recall

Adaptive Competitive Programming Pattern Retention System

Project Proposal – Software Engineering Course

Team Nexus 3

Subham Kumar (1024030665)

Tanush Grover (1024030666)

Shubhi Singh (1024030669)

August 2026

1. Project Overview / Motivation

Most students preparing for placements or competitive programming spend a large amount of time solving problems on platforms such as LeetCode. Over time this builds a big list of “solved” problems, but a solved count does not really tell a student whether they can recognize and apply the underlying technique when a similar-looking but different problem shows up later. It is fairly common to solve a problem like 3Sum using the Two Pointers technique, feel confident about it, and then struggle a few weeks later with a different Two Pointers problem because what got remembered was the specific solution, not the pattern behind it.

CP Recall is our attempt at building a system around this specific gap. Instead of only tracking which problems a student has completed, it tracks how the student is doing at the level of the underlying DSA pattern, and it schedules review in a way that actually tests whether the pattern has been retained — by giving the student a new problem from the same pattern family instead of the same question again. The idea driving the whole project can be summarized as: we revise the pattern, not the question.

This is a Software Engineering project, so our focus through this proposal is on designing a workable system — requirements, workflow, modular architecture, data handling and scheduling logic — rather than on inventing a new algorithm. Everything described below is meant to be realistic for a semester-long, 3rd-year project.

2. Problem Statement

The problem we are addressing can be broken down into the following points:

- Problem-solving platforms encourage students to solve a large number of questions, but the total count of problems solved does not necessarily mean the student will remember the technique later.
- Students often end up remembering the exact solution to a problem instead of understanding whether the same approach can be applied to a different problem using the same pattern.
- Most platforms do not give a simple, pattern-level view of which DSA patterns a student is actually strong or weak in — progress is shown mainly per-problem or per-topic tag, not per underlying technique.

- Because of this, students usually do not know which patterns are due for revision and when they should revisit them.
- Manually keeping track of revision schedules for a large number of patterns (arrays, two pointers, DFS/BFS, DP, sliding window, etc.) by hand is tedious and most students simply stop doing it after a while.
- Re-solving the exact same problem a second time is not a very strong test of whether the pattern was actually retained, since the student may just remember the earlier solution.
- There is a need for a lightweight system that tracks performance at the pattern level, schedules reviews based on that performance, and uses a different, suitable problem from the same pattern family to check retention rather than the original problem.

CP Recall is proposed as a practical response to these points, built as a companion tool that works alongside an existing platform (LeetCode) instead of trying to replace it.

3. Proposed Solution

3.1 Overview

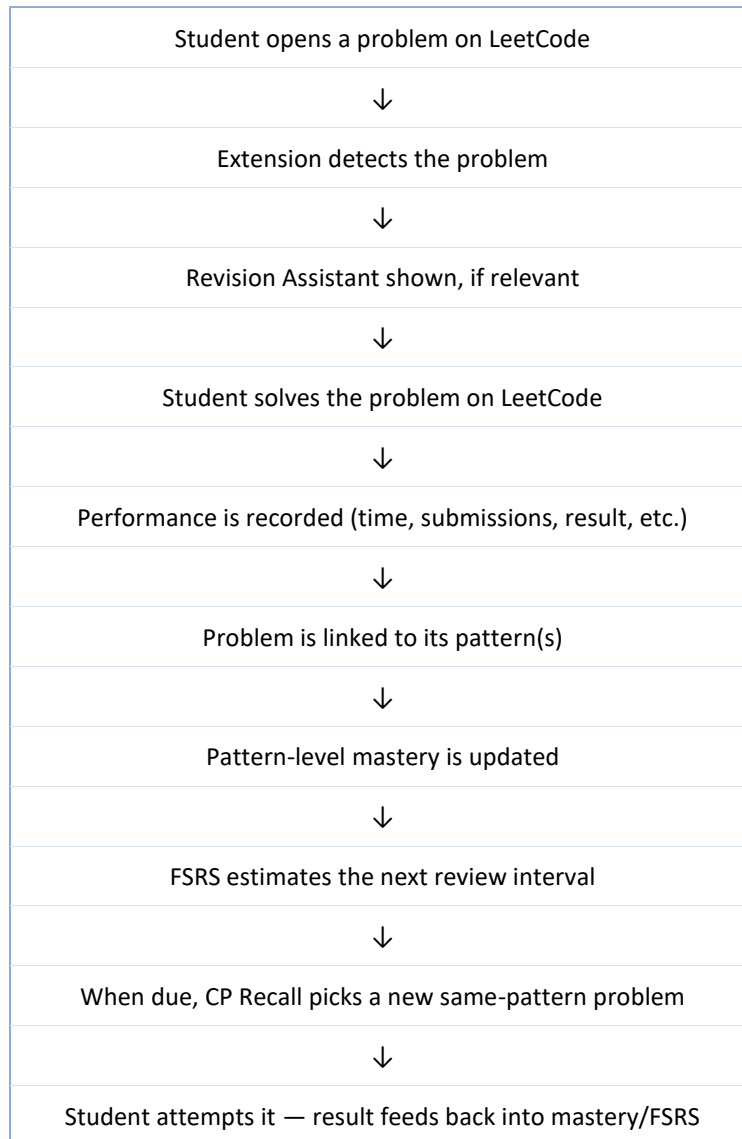
CP Recall is designed as a browser extension paired with a backend application. The extension works alongside LeetCode and can show a Revision Assistant interface when it is relevant to the problem the student currently has open. The backend keeps track of problems, patterns, the student's performance history, pattern-level mastery, and the review schedule. At a high level, the proposed solution includes the following pieces:

1. Detection of the current LeetCode problem through the browser extension.
2. A Revision Assistant interaction that surfaces relevant pattern/review information when appropriate.
3. Recording of problem information and identification of the relevant pattern(s).
4. Performance recording after an attempt (time taken, submissions, result, and similar signals).
5. Pattern-level mastery tracking that is updated as the student attempts more problems.
6. FSRS-based scheduling to estimate a reasonable next-review interval for each pattern.
7. A Problem Library for browsing and filtering problems.
8. A Home Dashboard giving an overview of overall progress.
9. A Pattern Dashboard focused on mastery per pattern.
10. Problem Details view for a single problem, including its review history.
11. Selection of a suitable, different problem from the same pattern family once that pattern becomes due for review.
12. Storage of review and performance history for future reference.

We are deliberately keeping the feature list close to this — the aim is a system that does pattern tracking and review scheduling well, not a system with a long list of unrelated features.

3.2 Core Workflow

The general flow of the system, from a student's point of view, looks like this:



The important distinction we keep in mind while designing this is that FSRS decides when a pattern should come up for review, while CP Recall's own logic decides what problem should actually be given for that review. These are two separate concerns and we are keeping them as separate modules so each can be tested on its own.

4. Solution Approach

4.1 LeetCode Browser Extension

CP Recall does not try to replace LeetCode — the student still solves the actual problem there. The browser extension's job is to detect which problem is currently open and, when it is relevant (for example, when the underlying pattern of that problem is close to its review point, or when the student would benefit from seeing their history with that pattern), provide a Revision Assistant interface alongside the problem. This interface is not meant to be a permanent sidebar sitting on the page at all times; it appears when it is useful and stays out of the way otherwise. Keeping the extension as a thin layer on top of the existing LeetCode workflow also makes it easier to build and test independently of the rest of the application.

4.2 Pattern Identification and Tracking

Every problem in our system is associated with one or more underlying patterns (for example, 3Sum is tagged under Two Pointers). Instead of storing just “problem solved: yes/no”, CP Recall stores this problem-pattern relationship and keeps performance information against the pattern as well as the problem. We plan to start with a curated set of problem-pattern mappings rather than trying to auto-detect the pattern of a problem, since automatically inferring the technique behind arbitrary code is a much harder problem than what this project needs to solve. The mapping can be extended over time.

4.3 Performance Review

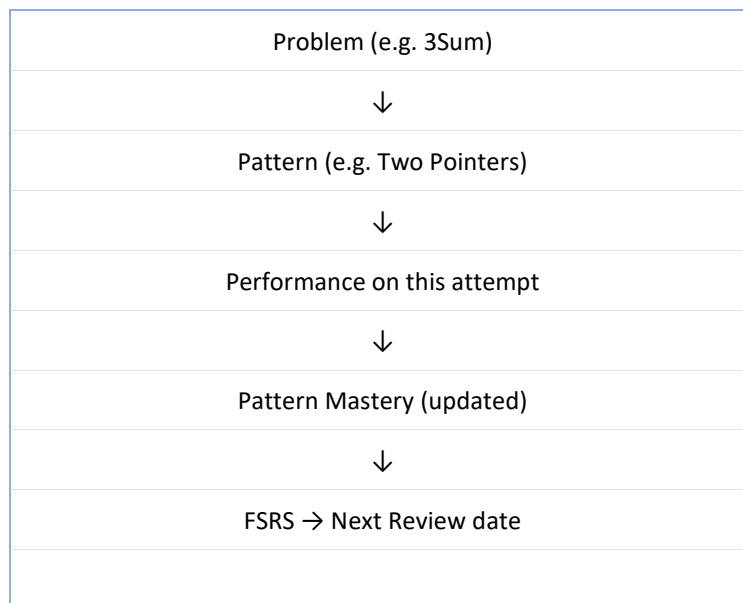
Once a student finishes attempting a problem, CP Recall records relevant signals about that attempt, such as:

- Time taken to solve the problem
- Number of submissions before an accepted result
- Final result (accepted / not accepted)
- Approximate solution efficiency / time complexity, where this can reasonably be determined
- Number of hints used, if the Revision Assistant provided any

We are not claiming that the system will automatically understand every aspect of a student's submitted code. The performance signals above are meant to be practical indicators, not a full code-analysis engine, and we will keep this in mind while deciding what is actually feasible to implement in a semester.

4.4 Pattern Mastery

Performance on individual attempts is aggregated into a mastery indicator for each pattern, shown roughly as a percentage. This is meant to be an evolving indicator that moves up or down as the student attempts more problems in that pattern, not an exact or scientifically precise measurement of what the student knows. A simplified view of how one attempt feeds into the system looks like this:



4.5 FSRS-based Scheduling

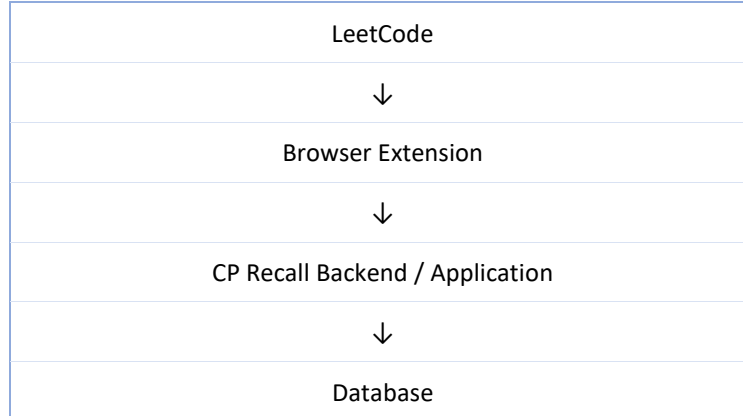
For deciding when a pattern should come up for review, we are using FSRS (Free Spaced Repetition Scheduler) instead of writing our own scheduling algorithm from scratch. In practical terms, FSRS is used to estimate an appropriate interval before the next review, based on the student's previous performance on that pattern. We are not claiming that FSRS perfectly predicts forgetting or guarantees retention — it is simply a reasonably well-established scheduling approach that gives us a sensible interval to work with, and it saves us from having to design and tune a spaced-repetition algorithm ourselves as part of this course project.

4.6 Problem Selection for Review

When a pattern becomes due according to FSRS, CP Recall does not simply resurface the same problem the student solved earlier. Instead, it selects a different problem that belongs to the same pattern family, from the Problem Library. The reasoning here is that if a student can solve a new problem using the same underlying technique, that is a much better indicator that the pattern has actually been learned, compared to just remembering a specific previous solution. Problem selection logic in the initial version can be fairly simple (e.g., pick an unattempted or less-recently-attempted problem tagged with the due pattern, at a similar or slightly higher difficulty), and can be improved in later iterations.

4.7 System Architecture

At a high level, the system is organized as follows:



Within the backend, we are planning the following logical modules:

- Authentication / User Management – handles student accounts and sessions
- Problem and Pattern Management – stores problems, patterns, and their relationships
- Performance Tracking – records attempt-level data
- Pattern Mastery – aggregates performance into a mastery indicator per pattern
- FSRS Scheduling – estimates next-review intervals per pattern
- Review / Problem Selection – picks a suitable same-pattern problem when a review is due
- Dashboard and Analytics – serves the data shown on the Home and Pattern dashboards

The exact technology stack will be finalized during implementation based on project requirements, but we intend to keep the frontend, backend, and browser extension as separate, loosely coupled components so that each can be developed and tested on its own.

4.8 Frontend Overview

Based on our current wireframe, the frontend is organized into four main views rather than a separate revision page — review-related information is shown inside the relevant dashboard or problem view instead of on its own screen.

Home Dashboard – gives an overview of the student's overall progress, such as problems solved, patterns due today, number of patterns tracked, and current streak. As an example, our wireframe shows values like 53 problems solved, 2 due today, 14 patterns tracked and a 7-day streak — these are illustrative numbers from the mock-up and not fixed targets.

Pattern Dashboard – focuses on mastery per pattern (for example Arrays, DFS, DP, Two Pointers), showing the pattern name, number of associated problems, mastery percentage, and which patterns currently look weak.

Problem Library – lets the student search and filter problems by pattern or status, and sort by fields such as mastery or next review date, so they can see their overall standing across problems at a glance.

Problem Details – shows information for one specific problem: its name, associated pattern(s)/tags, current mastery, review/attempt history, and next review date. As an example, our wireframe shows 3Sum tagged under Two Pointers with 76% mastery, an FSRS rating of “Good” and a next review date of Aug 20, 2026 — again, an illustrative value from the mock-up rather than a guaranteed figure.

5. Evaluation Criteria

Since the system has not been built yet, we cannot report actual results at the proposal stage. Instead, we are listing the metrics and methods we plan to use once an initial version is working:

5.1 Pattern retention

Whether a student can successfully solve a new problem belonging to a pattern that was recently reviewed. This is the most direct check of whether the retention idea behind CP Recall is actually working.

5.2 Review adherence

How often students actually attempt the problems that come up as due for review, versus ignoring them.

5.3 Mastery progression

Whether a pattern's mastery score trends upward over repeated attempts and reviews for a given student.

5.4 Review scheduling usefulness

Whether the FSRS-based intervals feel like a practical revision schedule in practice — for instance, whether students find the suggested review timing reasonable rather than too frequent or too sparse.

5.5 User usability

Feedback collected from a small group of student users covering ease of use, usefulness of the dashboards, clarity of the review information shown, and whether the pattern-level view is actually helpful compared to how they currently track their practice.

6. Scalability and Future Extension

6.1 Scalability

We are keeping scalability considerations realistic for a project of this scope:

- Modular separation between frontend, backend, and browser extension
- Indexed queries for problems and patterns, especially for filtering in the Problem Library
- Pagination in the Problem Library rather than loading every problem at once
- A largely stateless API design where possible, to keep the backend simple to scale
- Ability to add more patterns and problems to the dataset without structural changes
- Scheduling logic (FSRS integration) kept as its own module so it can be optimized independently if needed

6.2 Future Scope

The following are possibilities for later, beyond what we are planning to build this semester, and should be read as future ideas rather than committed features:

- Improved problem-selection/recommendation logic beyond the initial simple rules
- Richer pattern-level analytics and progress visualizations
- More detailed learning analytics over longer time periods
- Support for additional competitive-programming platforms beyond LeetCode
- Improved browser-extension capabilities
- Personalized difficulty selection for review problems

7. Project Scope and Deliverables

7.1 Initial Deliverable

For the first working version, we are keeping scope limited to a system that demonstrates the core idea end to end:

- User authentication
- Problem and pattern data management
- Basic browser extension prototype for LeetCode
- Revision Assistant interaction
- Performance recording after an attempt
- Pattern mastery tracking

- FSRS integration for review scheduling
- Home Dashboard
- Pattern Dashboard
- Problem Library
- Problem Details, including next-review information
- Basic testing of the above modules

7.2 Subsequent Deliverables

If time permits after the initial deliverable, we intend to work on:

- Improved problem-selection logic for reviews
- Richer analytics on the dashboards
- Better browser-extension integration and edge-case handling
- Additional pattern categories in the dataset
- General usability improvements based on early user feedback

8. Risks and Mitigations

8.1 Browser extension limitations

Browser extensions can be restricted by the platform they run on, or break if LeetCode changes its page structure. Mitigation: keep the extension's functionality modular and maintain a clear separation between LeetCode-page interaction and CP Recall's own application logic, so a change on one side does not require rewriting the other.

8.2 Problem-pattern mapping quality

Incorrect or incomplete pattern tags would affect mastery tracking and problem selection. Mitigation: start with a small, manually curated and validated set of problem-pattern mappings, and expand it gradually rather than trying to cover a large problem set immediately.

8.3 Mastery estimation limitations

A single mastery percentage cannot capture everything about a student's understanding. Mitigation: treat mastery as an evolving indicator meant to guide review scheduling, not as an absolute or final measure of skill.

8.4 FSRS integration complexity

Integrating an external scheduling algorithm correctly can be tricky, especially around edge cases with little performance history. Mitigation: wrap FSRS behind a well-defined internal module/interface and test that module independently of the rest of the system.

8.5 User adoption

Students may not bother using a tool that requires extra manual effort on top of just solving problems. Mitigation: keep the day-to-day workflow simple, avoid requiring students to manually enter information that can be inferred automatically, and keep the extension lightweight.

8.6 Evaluation limitations

With a small pilot group and limited time, evaluation results will be indicative rather than conclusive. Mitigation: use a small controlled group of student testers and clearly defined metrics (as listed in Section 5) so that whatever results we do get are at least measured consistently.

9. Summary / Conclusion

CP Recall proposes a practical way to address a gap we have both run into ourselves while practicing DSA — solving a lot of problems without necessarily retaining the patterns behind them. By tracking mastery at the pattern level, using FSRS to decide when a pattern is due for review, and testing that review with a different problem from the same pattern family rather than the original question, the system aims to give a more honest picture of what a student has actually retained. As a Software Engineering project, our focus through implementation will be on getting the workflow, data model, and module boundaries right, and on delivering a working initial version before attempting any of the extensions listed in Section 6.2. We revise the pattern, not the question — and that idea is what CP Recall is built around.